U.S. General Services Administration

Agentic AI Book Club - Session x

Reviewing Chapter 1.6 & 1.7A

US General Service Administration
AI Community of Practice - March 2026

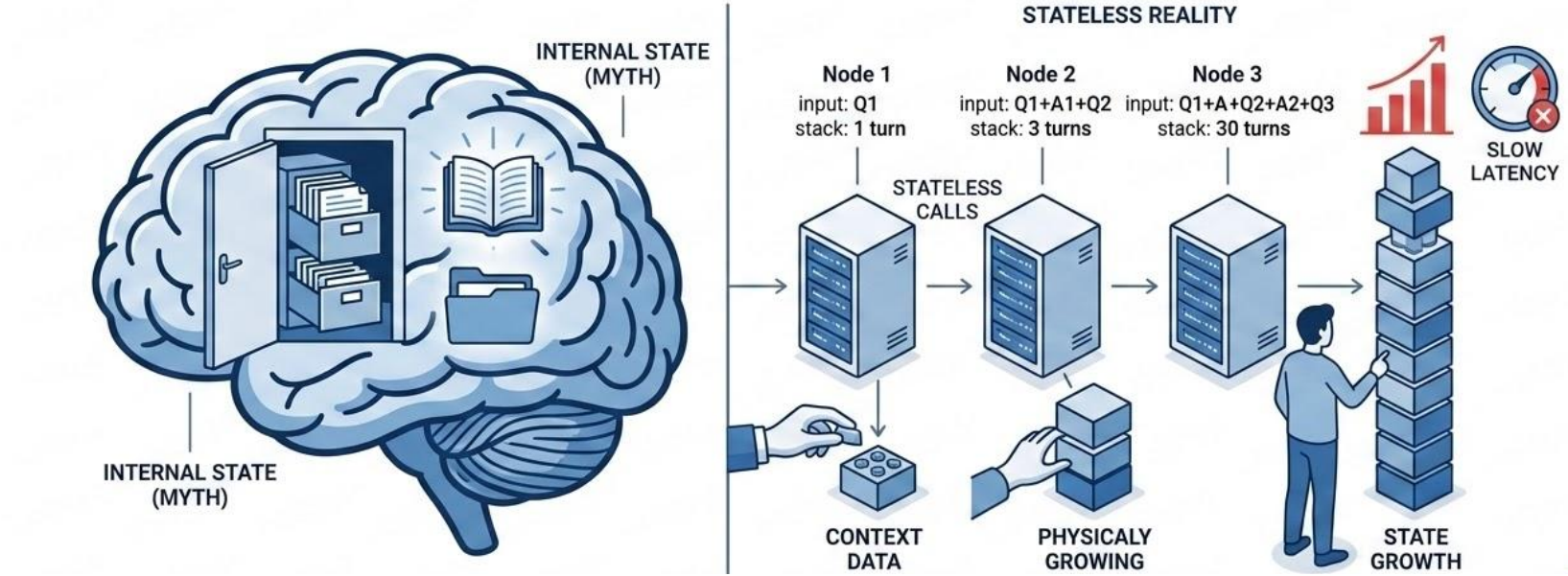
Agenda

- Housekeeping
- Chapter review
- Quiz review
- Intro to next week’s chapters

Housekeeping

- **Agentic AI is an early and rapidly evolving knowledge domain**
- **Tue Apr 14, 2026 1pm – 2pm (EDT)**
- 1 chapter behind in knowledge quiz
- Please keep doing reflection quiz - analysis coming towards the end of cohort 1 - section 1
- All texts for the chapters were sent and also available on Connect.gov

The Stateful Misconceptions



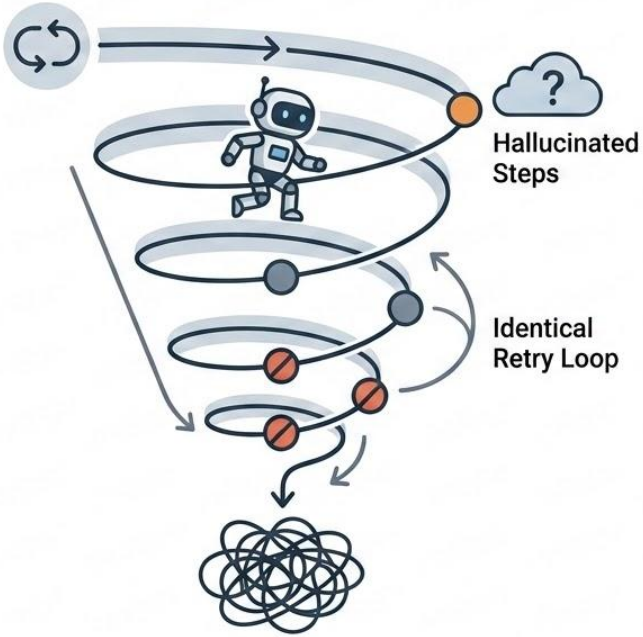
Quiz Review

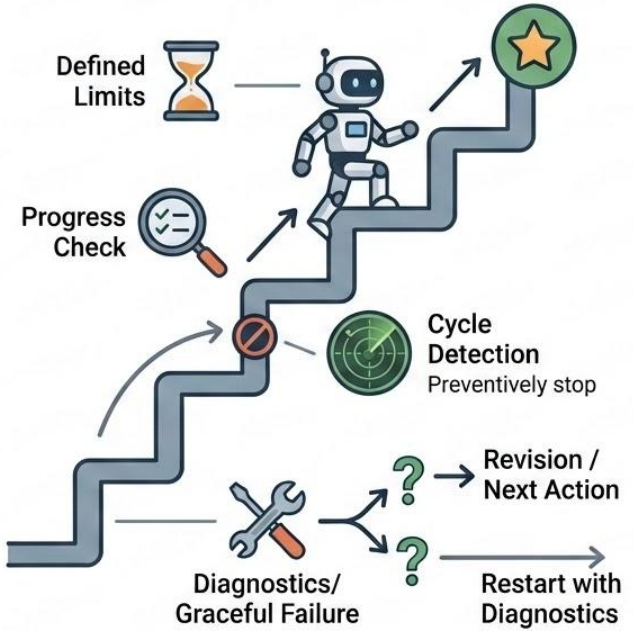
Design memory management for a customer service chatbot handling 100+ turn conversations about order history, account issues, and product questions. Balance information preservation with context efficiency.

A:

- **Detailed short-term memory + summarized long-term memory. Summarize in batches every 20 turns. Prioritize goal-relevant information in summaries.**
- Avoid summaries because they risk losing edge-case details; instead rely on a large context window and keep the full verbatim transcript, using retrieval only when the window is exceeded.
- Summarize after every turn so memory is always current; this prevents loss of details and is more efficient than periodic batching because each summary is small and incremental.
- Use simple truncation (e.g., keep the last 20 turns) because older content is rarely needed; treat all messages equally and don't maintain structured facts beyond the rolling window.
- **Maintain a structured "case file" updated when new facts appear, keep a small verbatim window for the latest interaction, and selectively retrieve original snippets only when needed.**

Infinite Loops and Hallucination Cycles



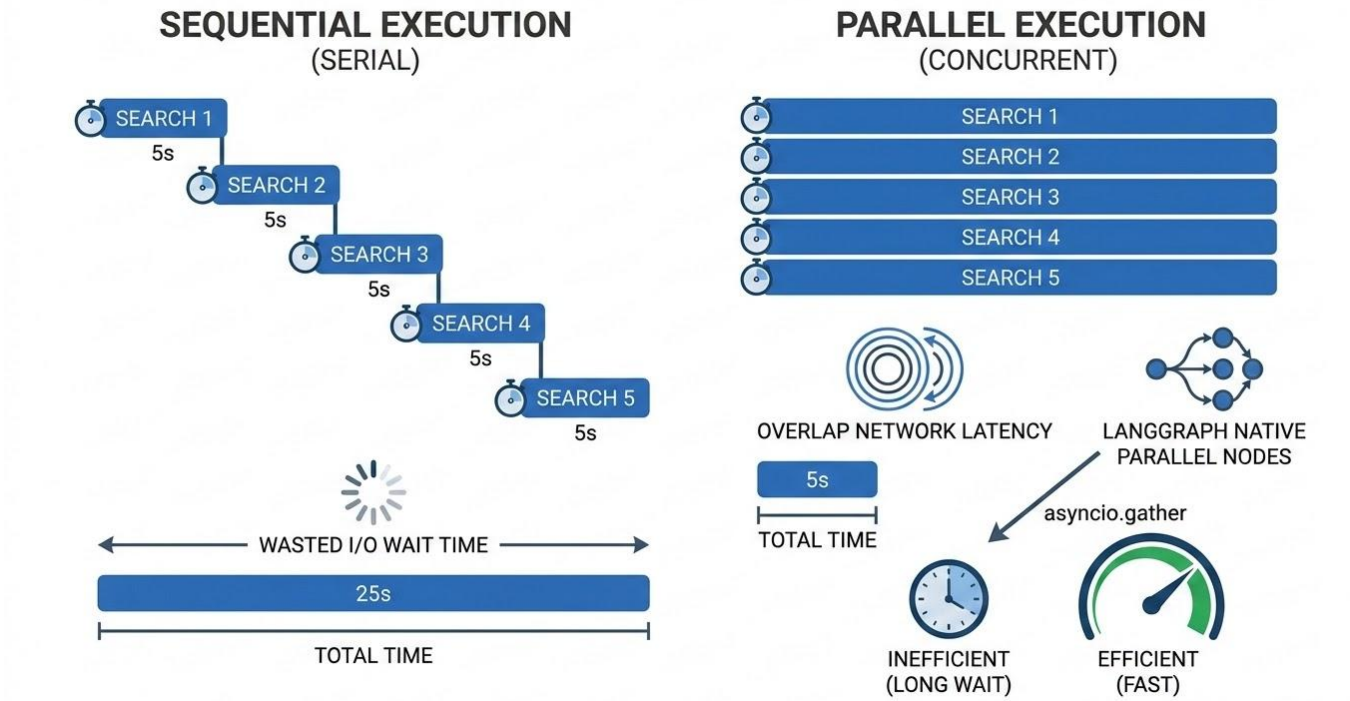


Hope is not a termination strategy

Three Layers of Loop Defense

- Hard iteration limits: ultimate safety net (5-20 steps)
- Progress validation: check if each step advances the goal
- State hashing: detect return to previously seen states
- Calibrate limits to workflow complexity
- Graceful failure with diagnostics beats silent spinning

Parallel Execution of Independent Operations



Quiz Review

A workflow needs to: (1) get user profile from database, (2) search user's order history, (3) check inventory for 3 specific items, (4) calculate shipping cost. Which operations can be parallelized?

A:

- **Operation 3 - Check inventory for 3 items.**
- All four operations can be parallelized.
- Parallelize steps 1, 2, 3.
- Not enough information for a decision

Note:

- Look for steps that do not need previous steps' outputs
- Pay attention to identical (sub-)tasks

Quiz Review

Chapter 1.5 quiz: A research workflow has three independent subtasks: (1) search academic papers, (2) check patent databases, and (3) query company websites. Each takes 30 seconds. How does explicit state management enable optimization here?

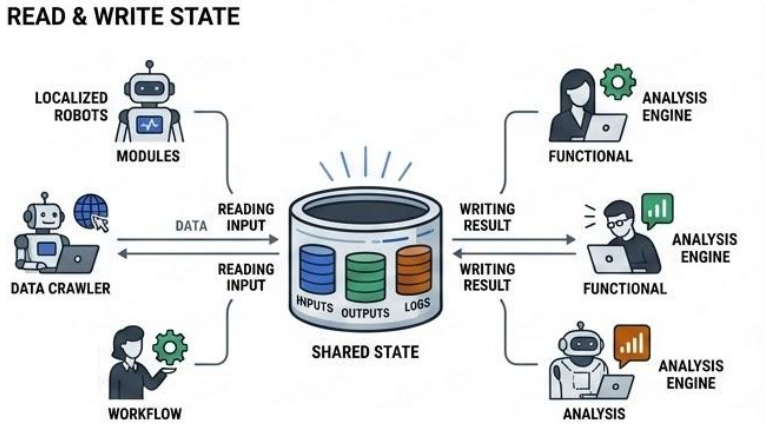
A:

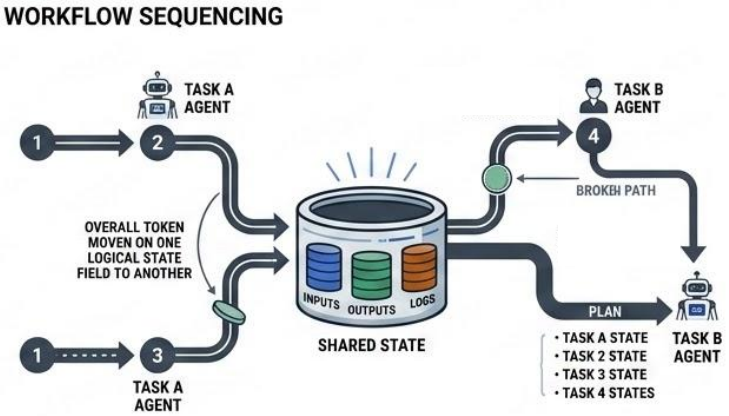
- **Explicit state management declares that the three subtasks are independent, so an orchestrator can run them in parallel.**
- Explicit state management alone is sufficient for parallelization because modern workflow engines automatically infer concurrency from the state schema.
- Explicit state management requires executing all subtasks sequentially so that the workflow engine can apply each state update in order and save time.
- Using function-local variables to track intermediate results to guarantee safe parallel execution, since the workflow's structure and step independence can be fully inferred.
- **Explicit state management defines shared inputs and outputs for each subtask, allowing the orchestrator to safely parallelize independent steps and then merge their results into a single workflow state.**

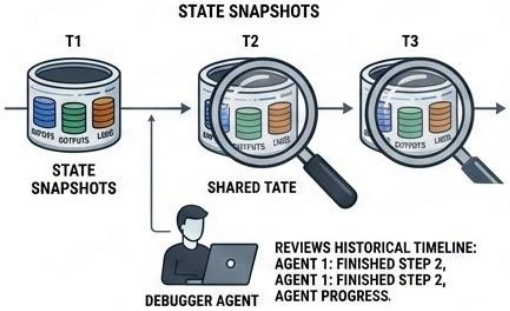
Stateful Orchestration for ReAct and P-n-E

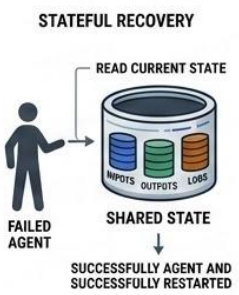
- ReAct depends on state to accumulate observations across cycles
- Plan-and-Execute needs state for plan tracking and replanning
- Execution failures trigger plan revision via state conditions
- State captures both planned and executed actions
- The pattern describes the cycle; orchestration implements it

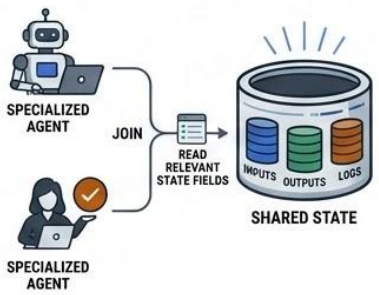
Multi-Agent Coordination Through Shared State











What problems must be solved by multi-agent system?

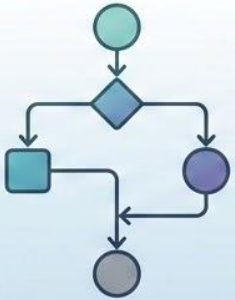
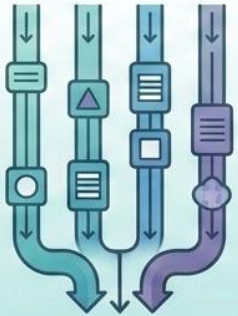
Intro to Local State vs. Distributed State

- Chapter 1.6: agent-local state in memory during execution
- Chapter 1.8: distributed state in Redis or PostgreSQL
- Same schemas and transformation logic, different persistence
- Distributed state enables horizontal scaling across instances
- New concerns: concurrent access, locking, retention policies

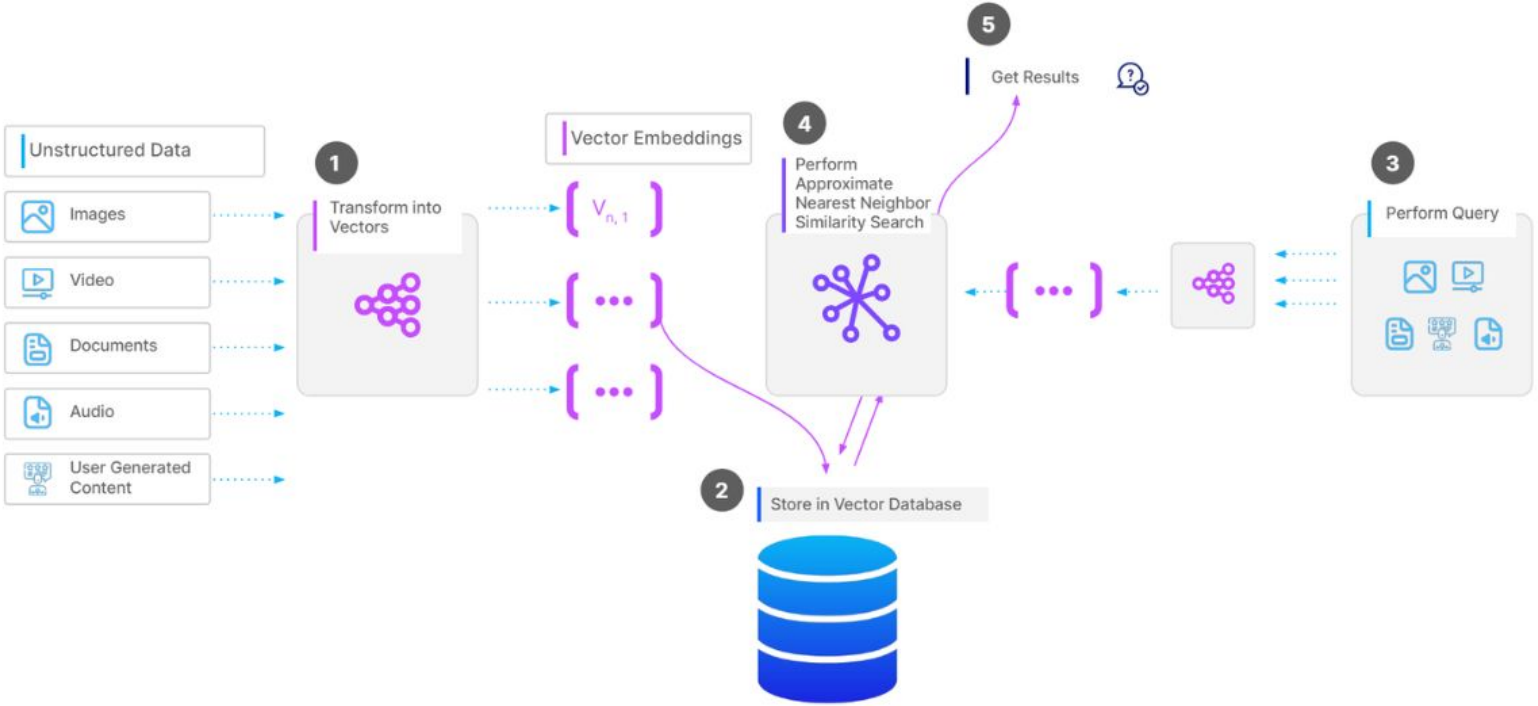
Foundations for Advanced Planning and Memory

- Hierarchical planning extends state machines with goal hierarchies
- Continual replanning adds monitoring loops to conditional transitions
- Memory systems distinguish working memory from long-term memory
- Working memory: current task state (what we built here)
- Long-term memory: patterns persisting across tasks (Chapter 1.4, Part 5)

Orchestration Decision Framework

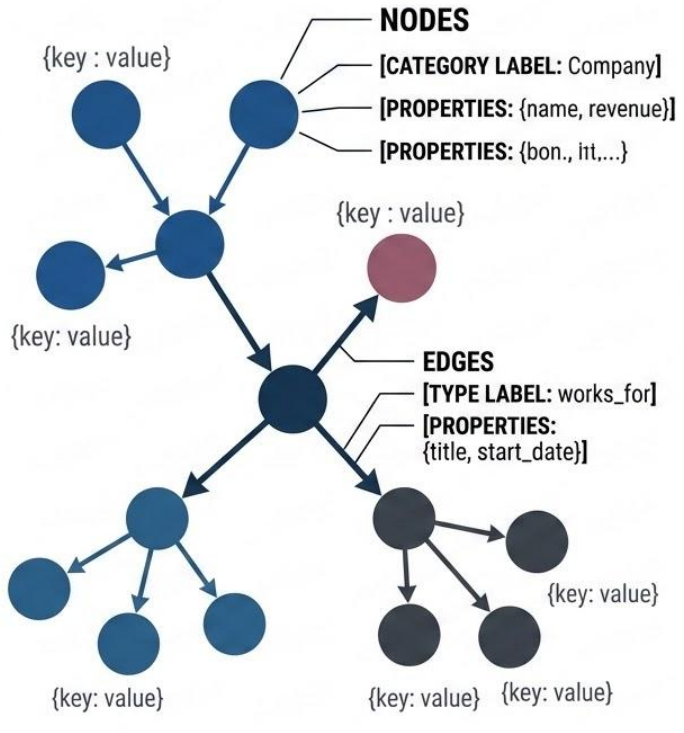
Match pattern complexity	Stateful orchestration	Graph-based routing	Parallel execution	Loop defense
				
FIT PATTERN TO WORKFLOW	STATE-BASED RECOVERY	VISIBLE BRANCHING LOGIC	PARALLEL EXECUTION	MANDATORY LOOP DEFENSE

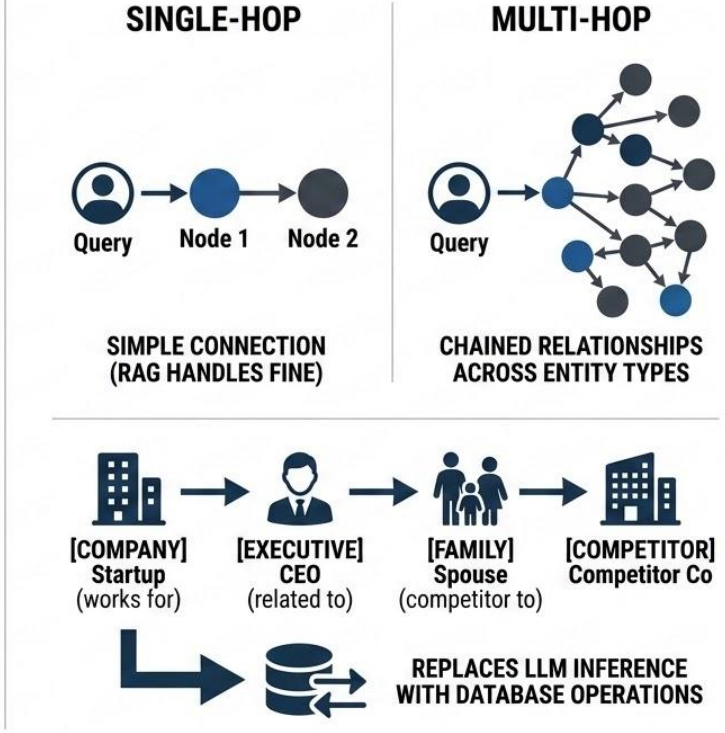
Review on Vector DB



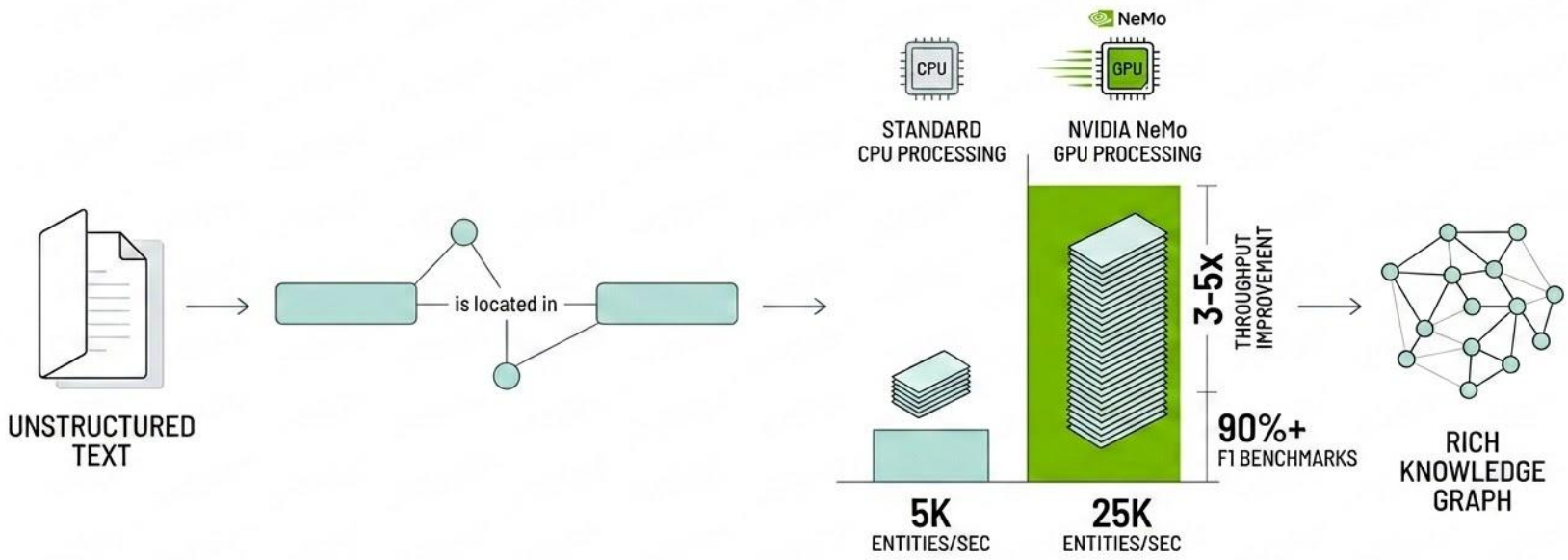
Credit: https://milvus.io/blog/what-is-a-vector-database.md

The Property Graph Model

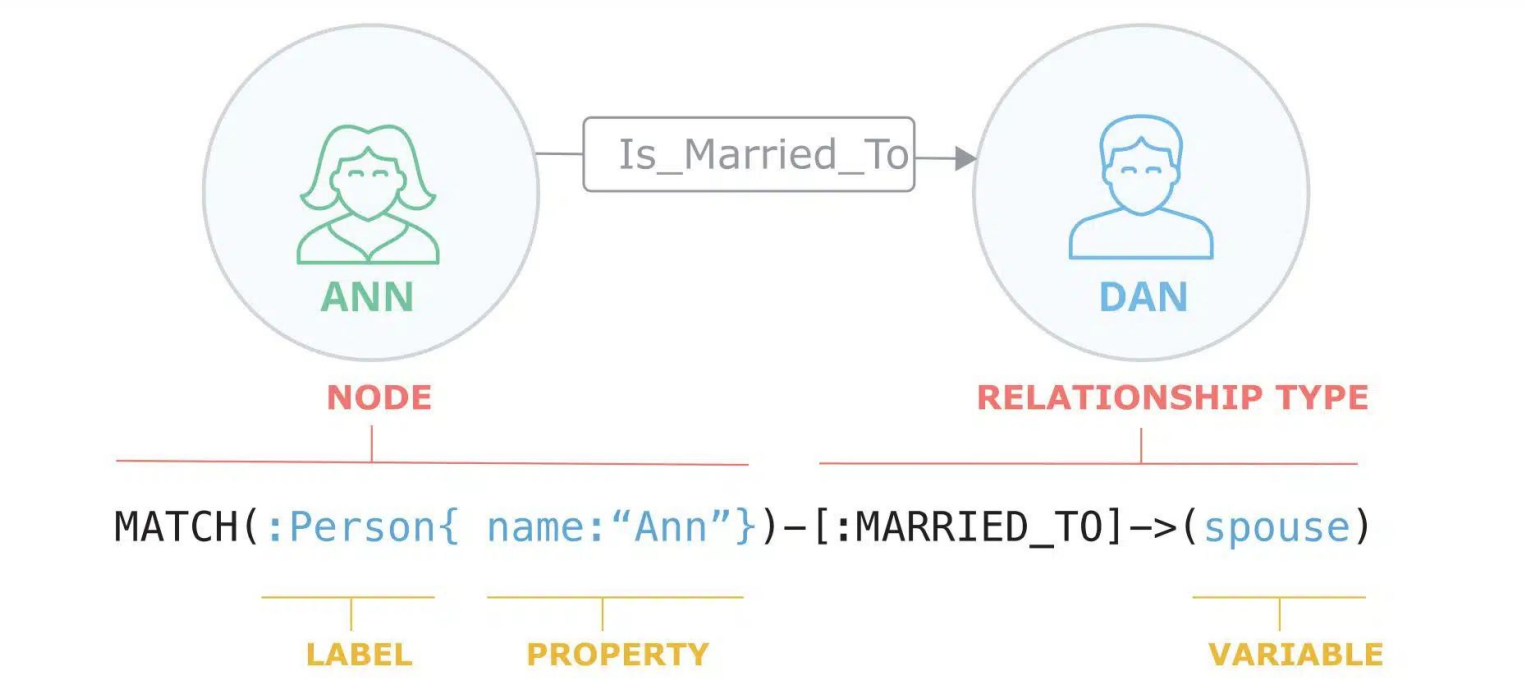




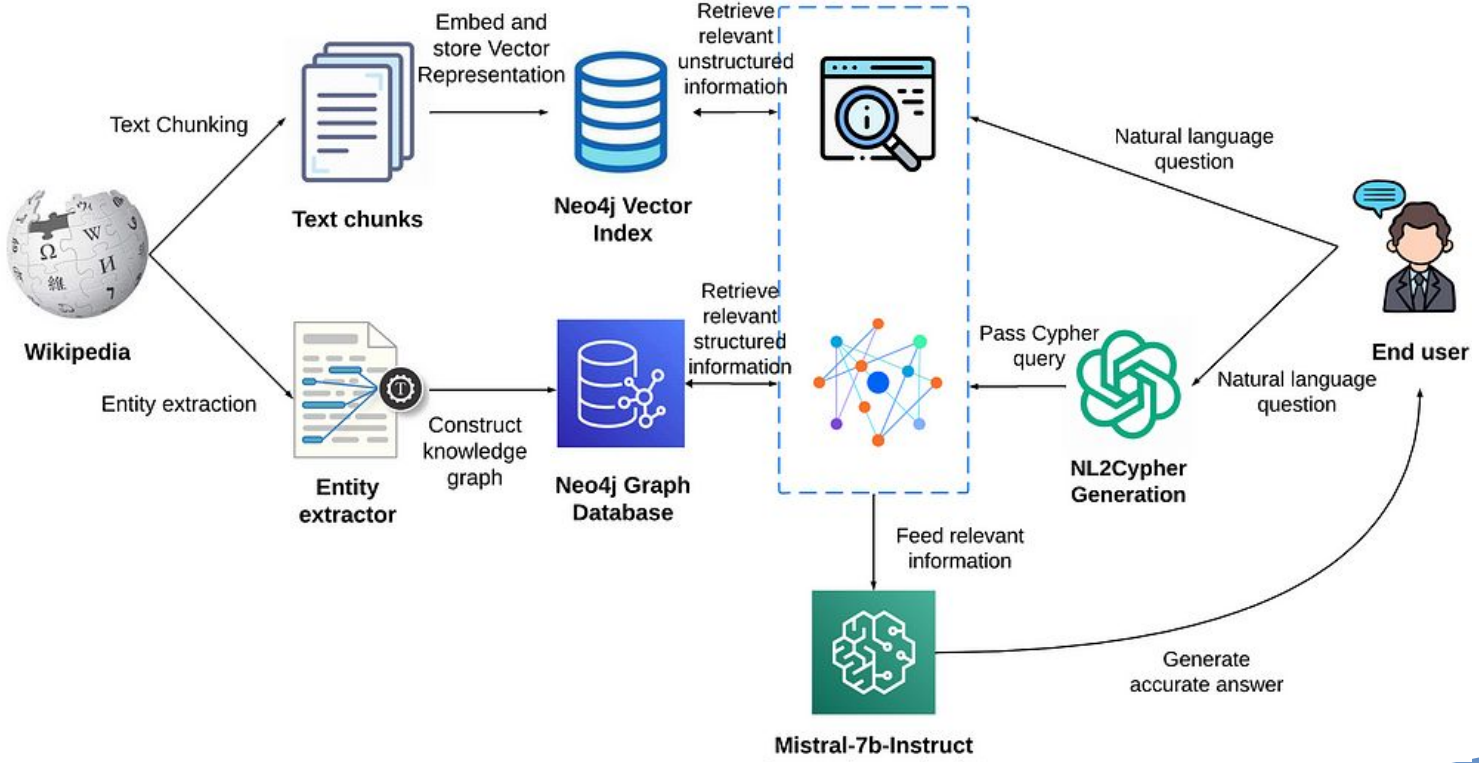
NER and Relationship Extraction in Depth



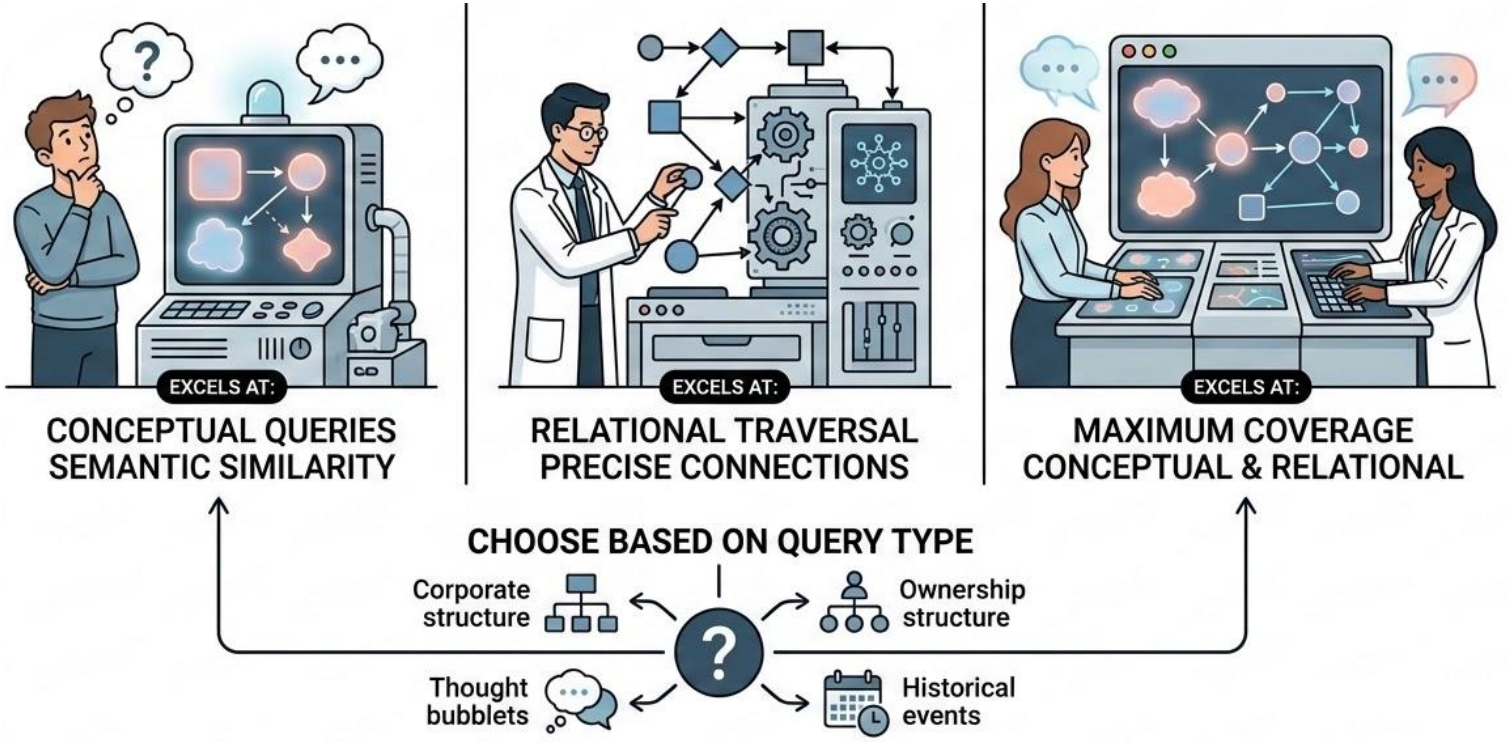
Cypher and Neo4j Fundamentals



LangChain GraphCypherQAChain Integration



RAG KG Hybrid



Next Chapters

- Part 1, Chapter 1.7B: Relational Reasoning with Knowledge Graphs - Hybrid RAG+KG Integration
  Addresses when and how to combine vector RAG with knowledge graph traversal through three hybrid integration patterns. Covers decision criteria distinguishing simple factual queries (RAG alone) from multi-hop relational queries (graph alone) and hybrid queries requiring both.

- Part 1, Chapter 1.8: Introduction to Scalability and Production Deployment
  This chapter discusses design and deploy agent systems that scale from single-GPU prototypes to enterprise deployments handling thousands of concurrent requests while optimizing for both cost and performance.

Preview